

Squelette minimal d'une IA

```
from typing import Literal
from ia import JeuDict, MoteurIA

class MonAI(MoteurIA):
    nom = "NOMFAMILLE1_NOMFAMILLE2" # A REMPLIR OBLIGATOIREMENT

    def __init__(self, joueur_id: str, ncases: int, max_tours: int,
temps_ko: int) -> None:
        pass

    def ponte(self, jeu: JeuDict, cout_ponte: int) -> Literal["OUV", "BOU",
"ECL", "RIEN"]:
        pass

    def action_abeilles(self, jeu: JeuDict) -> list[tuple[str, int, int,
Literal["DEPLACEMENT", "BUTINAGE"]]]:
        pass
```

Remplissage du nom de l'IA

Le nom donné à l'IA dans la variable `nom` doit être composé des noms de famille en majuscule de chaque étudiant du groupe séparés par un `_`. Si le nom de famille est en plusieurs mots, chaque mot doit être séparé par un tiret `-`. Pas d'accents ni autre caractères comme l'apostrophe.

Exemples:

```
nom = "NOM-FAMILLE1_NOMFAMILLE2"
nom = "MARTIN_DESJARDINS-FLEURY"
```

La fonction `__init__`

Elle est appelée une seule fois au début du jeu, elle vous donne certaines informations de base sur le jeu que je vous recommande de garder en mémoire. C'est également dans cette fonction que vous pouvez définir les variables que vous souhaitez garder tout au long du jeu et que vous pourrez réutiliser dans les deux autres fonctions.

La fonction `ponte`

Cette fonction est la première appelée au début de votre tour. Elle vous donne un dictionnaire `jeu` vous donnant toutes les informations de la partie auquel vous avez le droit. Vous avez également le paramètre `cout_ponte` qui est le montant de nectar nécessaire pour pondre une abeille. Cette fonction attends que vous retourniez une chaîne de caractère pouvant être:

- `"OUV"` : Si vous souhaitez pondre une ouvrière

- "BOU" : Si vous souhaitez pondre un bourdon
- "ECL" : Si vous souhaitez pondre une éclaireuse
- "RIEN" : Si vous souhaitez ne rien pondre pour ce tour

Info: Si vous retournez un type d'abeille mais que vous n'avez pas assez de nectar pour pondre, votre action sera ignorée.

La fonction `action_abeilles`

Cette fonction est appelée juste après `ponte()` dans votre tour. Elle vous redonne le dictionnaire de la partie mis à jour. Cette fonction attend que vous retourniez une liste d'actions pour vos abeilles, ces actions sont soit un butinage, soit un déplacement. Une action est écrite sous forme d'un tuple comme ceci:

```
action = ("123", 0, 1, "DEPLACEMENT")
action2 = ("456", 1, 2, "BUTINAGE")
```

L'action se décompose en 4 parties:

```
action = ("ID_ABEILLE", POSITION_X, POSITION_Y, "TYPE_ACTION")
```

Si on reprend les exemples précédents, ils peuvent se lire comme suit:

```
# On souhaite déplacer l'abeille nommée "123" en position (0, 1)
("123", 0, 1, "DEPLACEMENT")

# On souhaite que l'abeille nommée "456" butine la fleur située en position
(1, 2)
("456", 1, 2, "BUTINAGE")
```

Le retour de la fonction `action_abeilles()` est donc une liste de ces actions. Cela vous permet de choisir l'ordre des actions. Si vous souhaitez qu'une abeille ne fasse pas d'action pour ce tour il suffit de ne pas l'inclure dans la liste. Exemple plus complet:

```
return [ ("123", 0, 1, "DEPLACEMENT"), ("456", 1, 2, "BUTINAGE"), ("789",
1, 2, "BUTINAGE") ]
# Ici vous demandez à faire les actions suivantes (dans l'ordre):
# - Déplacer l'abeille "123" en position (0, 1)
# - Faire butiner l'abeille "456" sur la fleur en position (1, 2)
# - Faire butiner l'abeille "789" sur la fleur en position (1, 2)
```

Attention: il n'est attendu qu'UNE action PAR abeille. Une abeille ne peut pas se déplacer plus d'une fois, et elle ne peut pas se déplacer puis butiner ou inversement. Si vous le faites, la première action valide sera prise en compte et la deuxième sera ignorée.

Le dictionnaire de jeu

```
JeuDict = dict{
    "tour_actuel": int,
    "fleurs": list[dict{ "x": int, "y": int }],
    "moi": dict{
        "id": str,
        "position": dict{ "x": int, "y": int },
        "nectar": int
        "abeilles": list[dict{
            "abeille_type": "OUV" | "BOU" | "ECL",
            "id": str,
            "joueur_id": str,
            "force": int,
            "max_nectar": int,
            "position": dict{ "x": int, "y": int },
            "ko_temps": int,
            "nectar": int
        }]
    },
    "autres_joueurs": list[dict{
        "id": str,
        "position": dict{ "x": int, "y": int },
        "abeilles": list[dict{
            "abeille_type": "OUV" | "BOU" | "ECL",
            "id": str,
            "joueur_id": str,
            "force": int,
            "max_nectar": int,
            "position": dict{ "x": int, "y": int },
            "ko_temps": int
        }]
    }]
}
```

Quelques exemples d'utilisation:

```
# On affiche le tour actuel
print(jeu["tour_actuel"])

# On affiche la coordonnée X de la première fleur
print(jeu["fleurs"][0]["x"])

# On affiche son propre nectar
print(jeu["moi"]["nectar"])

# On affiche le nom et le nectar de chacune de mes abeilles
for abeille in jeu["moi"]["abeilles"]:
    print(abeille["id"], abeille["nectar"])
```

```
# On affiche le nom des autres joueurs
for joueur in jeu["autres_joueurs"]:
    print(joueur["id"])
```

/!\ ATTENTION: Ne partez pas du principe qu'il y aura toujours 4 joueurs. Le jeu pouvant se jouer de 2 à 4 joueurs, `jeu["autres_joueurs"]` peut contenir entre 1 et 3 joueurs.

Comment garder des informations en mémoire pendant la partie

Si vous souhaitez garder des informations qui doivent persister pendant toute la partie, que vous pouvez réutiliser dans vos fonctions vous devez les déclarer comme suit:

```
class MonAI(MoteurIA):

    def __init__(self, joueur_id: str, ncases: int, max_tours: int,
temps_ko: int) -> None:
        # Ici on déclare nos variables pour qu'elles soit utilisables dans
        les autres fonctions.
        self.ma_variable = 5
        self.mon_dictionnaire = { "macle": 10 }

    def ponte(self, jeu: JeuDict, cout_ponte: int) -> Literal["OUV", "BOU",
"ECL", "RIEN"]:
        self.ma_variable += 1
        self.mon_dictionnaire["macle"] += 10

        print(self.ma_variable)
        # Affiche 6 au premier appel de ponte(), puis 7 au deuxième appel,
        puis 8, etc...

        print(self.mon_dictionnaire["macle"])
        # Affiche 20 au premier appel de ponte(), puis 30 au deuxième
        appel, puis 40, etc...
```

ATTENTION A NE PAS FAIRE:

```
class MonAI(MoteurIA):
    ma_variable = 5 # /!\ Interdit : les variables doivent être déclarées
    dans le __init__()

    def __init__(self, joueur_id: str, ncases: int, max_tours: int,
temps_ko: int) -> None:
        pass

    def ponte(self, jeu: JeuDict, cout_ponte: int) -> Literal["OUV", "BOU",
"ECL", "RIEN"]:
        # Vous n'avez aucune garantie que votre variable persistera
        correctement
```

```
# Si elle n'est pas déclarée dans le __init__()
print(self.ma_variable)
```

Rajouter des fonctions

Vous pouvez rajouter des fonctions, et je vous le recommande pour faciliter l'écriture de votre IA:

```
class MonAI(MoteurIA):

    def __init__(self, joueur_id: str, ncases: int, max_tours: int,
temps_ko: int) -> None:
        self.ma_variable = 5

    # Le premier argument doit toujours être self
    def ma_fonction_perso(self, argument1: int) -> int:
        return self.ma_variable + argument1

    def ponte(self, jeu: JeuDict, cout_ponte: int) -> Literal["OUV", "BOU",
"ECL", "RIEN"]:
        variable_locale = self.ma_fonction_perso(10)

        print(variable_locale) # Affiche 15
```

Quelques règles globales au fichier d'IA

Vous êtes autorisés à mettre du code en dehors de votre classe IA, mais c'est limité, quelques exemples:

```
from typing import Literal
from ia import JeuDict, MoteurIA

# Définition de constantes: OK
MA_CONSTANTE = 10

# Définition d'autres classes: OK
class MaClasse:
    pass

# Définition de fonctions: OK
# Attention néanmoins, les fonctions écrites ici ne peuvent pas bénéficier
de l'argument self
# et donc n'auront pas accès aux variables déclarées dans le __init__().
def ma_fonction_perso(argument1):
    pass

# Définition de variables persistantes: INTERDIT
# Vous n'avez aucune garantie que votre variable persistera correctement
ma_variable = 10

# Des instructions de code directement: INTERDIT
```

```
print("Hello World!")
if MA_CONSTANTE == 5:
    print("TEST")

class MonAI(MoteurIA):
    nom = "NOMFAMILLE1_NOMFAMILLE2"

    def __init__(self, joueur_id: str, ncases: int, max_tours: int,
temps_ko: int) -> None:
        pass

    def ponte(self, jeu: JeuDict, cout_ponte: int) -> Literal["OUV", "BOU",
"ECL", "RIEN"]:
        ma_variable += 10 # INTERDIT

    def action_abeilles(self, jeu: JeuDict) -> list[tuple[str, int, int,
Literal["DEPLACEMENT", "BUTINAGE"]]]:
        pass
```

Astuce: Si vous souhaitez vérifier la conformité de votre fichier IA, enregistrez le auprès du bot discord. Il vérifie déjà un bon nombre de ces règles. Si le bot accepte votre IA c'est que vous êtes bon !